

# Tetris OS Simulator: A Custom Operating System Module Demonstration Using Game Engine Architecture

Mehak Jain, Anuj Kumar Singh  
Operating System

**Abstract**—This paper presents the design, implementation, and evaluation of Tetris OS Simulator — a fully-functional Tetris game engine built from scratch in C without standard library dependencies for core logic. The project simulates seven fundamental operating system modules: Process Management, Memory Management, File Systems, I/O Management, Error Handling & Security, Networking, and User Interface. The engine runs in two modes — a terminal-based interface using ANSI escape codes and a WebSocket-backed browser client using React. Six custom libraries (memory.c, string.c, math.c, screen.c, keyboard.c, sound.c) replace all standard library functionality, while a hardware abstraction layer confines POSIX networking to dedicated I/O modules. The system demonstrates arena-based dynamic memory allocation, persistent file-based leaderboards, real-time WebSocket communication with automatic reconnection, and comprehensive error handling across all subsystems.

**Index Terms**—Operating Systems, Custom Libraries, Memory Management, WebSocket, Tetris, Process Scheduling, Hardware Abstraction

## I. Introduction

### A. Problem Statement

Standard game engines rely heavily on standard library functions (malloc, strlen, sin, etc.) and opaque frameworks. This project challenges that paradigm by implementing a complete game system where all core logic is built from first principles — custom arithmetic, custom string operations, custom memory allocation, and custom I/O drivers. The complete source code and full commit history are publicly available at <https://github.com/Mehak261124/Tetris>.

### B. Objectives

- 1) Implement six custom foundational libraries replacing `<string.h>`, `<math.h>`, and `malloc/free`.
- 2) Build a complete Tetris game engine demonstrating all seven OS module categories.
- 3) Provide dual I/O modes: terminal (ANSI) and browser (WebSocket).
- 4) Ensure robust error handling, edge-case coverage, and graceful degradation.
- 5) Maintain persistent state across sessions (file system module).
- 6) Support real-time client-server communication (networking module).

### C. Scope

The project covers the complete lifecycle of an OS-like system: boot (memory initialization), process scheduling (game loop), I/O device management (keyboard/screen/speaker), persistent storage (leaderboard files), and graceful shutdown (memory cleanup, socket release).

## II. System Architecture

The system follows a layered architecture where each custom library represents an OS module:

Listing 1: System Architecture Layers

|                    |                                    |
|--------------------|------------------------------------|
| Application Layer  | : ws_tetris.c / main.c             |
| Core Library Layer | : string.c   math.c   memory.c     |
| HAL Layer          | : screen.c   keyboard.c   sound.c  |
| Host OS / Hardware | : POSIX sockets   Terminal   Audio |

**Design Principle:** Game logic never includes networking headers, POSIX headers, or standard library functions beyond `<stdio.h>` and `<stdlib.h>`. All I/O flows through the Hardware Abstraction Layer (HAL).

## III. Methodology: Seven OS Modules

### A. Module 1: Process Management

Process management determines who gets to run, when, and where. The game loop acts as a cooperative scheduler executing three processes each tick:

TABLE I: Process Schedule

| Process         | Priority | Frequency         |
|-----------------|----------|-------------------|
| Input Polling   | Highest  | Every tick (50ms) |
| Physics/Gravity | Medium   | Level-dependent   |
| Rendering       | Lowest   | On state change   |

**Speed Curve (NES-style):**

- Level 1–8:  $\text{drop\_ticks} = \max(13 - \text{level}, 5)$  — 600ms to 250ms
- Level 9–13:  $\text{drop\_ticks} = \max(8 - (\text{level}-8)/2, 2)$  — 150ms to 100ms
- Level 14+:  $\text{drop\_ticks} = 1$  — 50ms (maximum speed)

Lock Delay: A 500ms timer prevents instant locking when a piece touches the ground. The timer resets on successful moves or rotations, capped at 15 resets per piece to prevent infinite stalling.

DAS/ARR: The web client implements Delayed Auto Shift (170ms delay) followed by Auto Repeat Rate (50ms interval) for smooth, responsive key repeat matching modern Tetris standards.

Signal Handling: SIGINT and SIGTERM are intercepted by a lightweight handler that sets a volatile `sig_atomic_t` flag. The game loop checks this flag once per frame and exits through the normal teardown path, ensuring `keyboard_restore()` and `memory_cleanup()` always run — the terminal is never left in raw mode after a forced quit. SIGWINCH triggers a debounced resize handler: an 80 ms `usleep()` wait discards all intermediate signals fired during continuous window drag, then a single `screen_clear()` with the `\033[3J\033[2J\033[H` sequence erases iTerm2’s saved scrollbar lines before redrawing the full frame cleanly.

## B. Module 2: Memory Management

Memory management determines who owns which memory and how much.

Design: First-Fit Free-List Allocator with Block Splitting & Coalescing.

Listing 2: BlockHeader Structure

```
typedef struct BlockHeader {
    int size;
    int is_free;
    struct BlockHeader *next;
} BlockHeader;
```

Operations:

- `memory_init(1 048 576)` — acquires one 1 MB slab via the single permitted `malloc()` call.
- `t_alloc(size)` — first-fit search with 8-byte alignment; splits oversized blocks to minimise waste.
- `t_dealloc(ptr)` — validates pointer ownership by walking the block list, marks free, then coalesces adjacent free blocks in a single pass.
- `memory_cleanup()` — releases the entire slab back to the host OS via the single permitted `free()` call.

Safety guarantees: 8-byte alignment prevents misaligned struct access; pointer validation rejects bad inputs without crashing; double-free is silently ignored; NULL is returned on allocation failure so callers can check before use.

## C. Module 3: File System

The file system provides persistent storage for leaderboard history across sessions.

- Shared leaderboard file (`leaderboard.txt`) — both the terminal build (`main.c`) and the WebSocket backend (`ws_tetris.c`) read and write the same file using the

same format, so scores are shared across both play modes.

- Unified format: `score|level|lines|name` (pipe-delimited, one entry per line, sorted descending by score). Top 5 entries stored in terminal mode, top 10 in web mode.
- Parsing: `t_split()` from `string.c` tokenises each line — no `strtok` or `sscanf`.
- Edge cases: Missing file starts with an empty table (not an error); malformed lines are silently skipped; write failures degrade gracefully (game continues, score is not persisted).

## D. Module 4: I/O Management

I/O management defines how this computer talks to the world — screen, keyboard, and speaker.

TABLE II: I/O Device Mapping

| Device   | Terminal                   | WebSocket                       |
|----------|----------------------------|---------------------------------|
| Keyboard | <code>getchar()</code> raw | <code>recv()</code> + WS decode |
| Screen   | ANSI escape sequences      | WebSocket text frames           |
| Speaker  | WAV + platform player      | Web Audio API                   |

Audio — cross-platform design: `sound.c` generates square-wave WAV files at startup using pure integer arithmetic (no `<math.h>`). Playback uses a compile-time platform detection macro: `afplay` on macOS and `aplay` (ALSA) on Linux, with `paplay` (PulseAudio) as a secondary Linux fallback. A runtime availability check using `access()` confirms the binary exists before any `fork()` is attempted — if no player is found the game runs silently with no crash or zombie processes.

WebSocket I/O: `keyboard.c` owns the receive path (`keyboard_recv_ws` decodes RFC6455 masked frames). `screen.c` owns the send path (`screen_send_ws` encodes WebSocket text frames). Game logic in `ws_tetris.c` calls only these abstractions — zero POSIX socket headers appear outside the HAL.

## E. Module 5: Error Handling & Security

Error handling ensures graceful behaviour when things go wrong.

TABLE III: Error Handling Matrix

| Error Category             | Handler                                          |
|----------------------------|--------------------------------------------------|
| Division by zero           | <code>t_div/t_mod</code> return 0 safely         |
| Memory OOM                 | <code>t_alloc</code> returns NULL; caller checks |
| Board out-of-bounds        | <code>t_in_bounds()</code> guards all writes     |
| Buffer overflow            | <code>t_strncpy()</code> enforces max-length     |
| Double-free                | Block-list walk detects and ignores              |
| Invalid pointer free       | Range check + ownership walk rejects it          |
| SIGINT / SIGTERM           | Flag set; loop exits; terminal restored          |
| SIGWINCH                   | Flag set; full redraw on next frame              |
| Client disconnect          | Server re-accepts without restart                |
| Malformed JSON / WebSocket | Frame silently dropped; loop continues           |
| Handshake failure          | Retry with next client connection                |
| Audio player missing       | Runtime check; game runs silently                |

Secure halt: `screen_panic(msg)` is called on unrecoverable errors (e.g. allocation failure at boot). It restores the

terminal cursor and raw-mode state, prints a [KERNEL PANIC] message to stderr, and calls exit(1) — the shell is always left in a usable state.

#### F. Module 6: Networking

Networking defines how this computer talks to another computer.

Protocol: WebSocket (RFC 6455) over TCP on port 8080.

Key features:

- Persistent server socket: survives page refresh and client reconnection. The outer accept loop re-accepts a new client without restarting the process.
- Auto-reconnect client: the React frontend retries with 1.5s exponential backoff on disconnect.
- Full RFC 6455 compliance: SHA-1 handshake (implemented without <openssl>), masked frame decoding, FIN bit handling, close frame detection.
- JSON state protocol: full game state serialised every dirty tick (board, current/ghost/next/held pieces, score, leaderboard). Serialisation uses only t\_itoa and manual string building — no sprintf or <string.h>.
- Hardware abstraction: all socket and POSIX network headers (sys/socket.h, arpa/inet.h) are confined exclusively to screen.c and keyboard.c — zero networking code appears in game logic.

#### G. Module 7: User Interface

The user interface defines how a user interacts with the game.

Terminal UI: ANSI-colored 10×20 board with piece-type colors, ghost piece (shown as --), next piece preview, hold piece display, HUD panel with score/level/lines/-combo, game-over overlay showing the top-5 leaderboard, player name entry via readLine(), 4 block themes switchable with T, and background music.

Web UI: React 18 + Vite with CRT scanline aesthetic, 4 selectable color themes (Neon, Retro, Mono, Pastel), hold piece, ghost piece, 7-bag randomizer, full SRS wall kicks (5-test per rotation), lock delay, DAS/ARR, combo system with live counter, level-up flash animation, top-10 leaderboard tab, mobile touch controls, SFX and music toggles, and auto-reconnect status indicator.

### IV. Implementation Details

#### A. Custom Library Pipeline

The six libraries form an explicit data-flow pipeline used in every game action:

Listing 3: Data Flow Pipeline

```
Input -> keyboard.c (keyPressed /
keyboard_rcv_ws)
Parse -> string.c (t_strcmp, t_split, t_strlen)
State -> memory.c (t_alloc / t_dealloc)
Logic -> math.c (t_mul, t_mod, t_div,
t_in_bounds)
```

```
Output -> screen.c (ANSI escapes /
screen_send_ws)
Audio -> sound.c (WAV gen / fork+exec player)
```

Concrete example — key dispatch: keyPressed() returns a char; key\_matches() wraps it in a 2-byte string and calls t\_strcmp() for comparison; the matched action calls t\_in\_bounds() before any board write; the updated score is converted with t\_itoa() and sent via screen\_render\_string(). Every layer of the pipeline is exercised in a single keypress.

#### B. Gameplay Mechanics

TABLE IV: Gameplay Feature Matrix

| Feature           | Terminal       | Web            |
|-------------------|----------------|----------------|
| 7 Tetrominoes     | ✓              | ✓              |
| Ghost Piece       | ✓              | ✓              |
| Soft / Hard Drop  | ✓              | ✓              |
| Level Progression | ✓              | ✓              |
| Sound Effects     | ✓              | ✓              |
| Background Music  | ✓              | ✓              |
| Player Name Entry | ✓              | ✓              |
| Score Persistence | ✓              | ✓              |
| Pause / Resume    | ✓              | ✓              |
| Hold Piece        | ✓              | ✓              |
| 7-Bag Randomizer  | ✓              | ✓              |
| Lock Delay        | ✓              | ✓              |
| Combo System      | ✓              | ✓              |
| Leaderboard       | Top 5          | Top 10         |
| Theme Switcher    | 4 block styles | 4 color themes |
| SRS Wall Kicks    | Basic (1-test) | Full (5-test)  |
| DAS / ARR         | —              | ✓              |
| Touch Controls    | —              | ✓              |

#### C. Technology Stack

TABLE V: Technology Components

| Component         | Technology                  |
|-------------------|-----------------------------|
| Terminal backend  | C (gcc -Wall -Wextra -O2)   |
| WebSocket backend | C (gcc -Wall -Wextra -O2)   |
| Frontend          | React 18 + Vite             |
| Transport         | WebSocket RFC 6455 / TCP    |
| Audio (macOS)     | WAV + afplay                |
| Audio (Linux)     | WAV + aplay / paplay        |
| Audio (Web)       | Web Audio API oscillators   |
| Persistence       | Plain text (pipe-delimited) |
| Build             | GNU Make / npm              |

### V. Results and Testing

#### A. Compilation

Both build targets compile with zero warnings and zero errors under strict flags:

- Terminal: gcc -Wall -Wextra -O2 src/\*.c -o tetris\_os
- Backend: gcc -Wall -Wextra -O2 backend/ws\_tetris.c src/{string,math,memory,screen,keyboard}.c -o tetris\_ws

## B. Demo

A screen recording demonstrating both the terminal and web modes is available in the project repository at <https://github.com/Mehak261124/Tetris>. A screen recording demonstrating both modes is available in the `/demo/` folder of the repository.

## C. Edge Cases Verified

TABLE VI: Test Results

| Test Case                    | Expected            | Result |
|------------------------------|---------------------|--------|
| Division by zero             | Returns 0           | Pass   |
| <code>t_alloc(0)</code>      | Returns NULL        | Pass   |
| <code>t_alloc OOM</code>     | Returns NULL        | Pass   |
| <code>t_dealloc(NULL)</code> | No-op               | Pass   |
| Double-free                  | Silently ignored    | Pass   |
| Invalid pointer free         | Rejected            | Pass   |
| SIGINT during gameplay       | Terminal restored   | Pass   |
| SIGTERM during gameplay      | Terminal restored   | Pass   |
| Terminal resize (SIGWINCH)   | Full redraw         | Pass   |
| Terminal too small           | Warning displayed   | Pass   |
| Page refresh (WS disconnect) | Re-accepts          | Pass   |
| Empty leaderboard file       | Starts fresh        | Pass   |
| Malformed leaderboard line   | Line skipped        | Pass   |
| Name overflow (>20 chars)    | Truncated safely    | Pass   |
| Empty name input             | Defaults to Player  | Pass   |
| Wall kick rotation (5 tests) | All 5 attempted     | Pass   |
| Hold twice per turn          | Second hold blocked | Pass   |
| Malformed WebSocket frame    | Silently dropped    | Pass   |
| WS handshake failure         | Retry next client   | Pass   |
| Audio player not installed   | Silent, no crash    | Pass   |

## D. Performance

- Game loop tick rate: 50 ms (20 Hz physics), 16 ms render budget ( 60fps)
- WebSocket frame size: 2–4 KB per state update
- Memory footprint: 1 MB arena allocated; game state occupies <2KB
- Frontend bundle: <100 KB (production build)

## VI. Known Issues

Honest self-evaluation of current limitations:

- T-spin detection not implemented. The `last_was_rotate` flag is tracked in `ws_tetris.c` but the 4-corner detection logic and T-spin bonus scoring are not yet applied.
- Terminal leaderboard limited to 5 entries. The web backend stores 10 entries; the terminal mode stores 5. Both read the same file so the terminal silently discards entries 6–10 on load.
- Background music does not loop seamlessly in the web client. The melody restarts with a brief gap between repeats due to the `setTimeout`-based sequencer.
- No DAS/ARR in terminal mode. Rapid key repeat relies entirely on the OS keyboard repeat rate; the 170 ms / 50 ms DAS/ARR system is web-only.
- Audio requires external binary on Linux. `aplay` (ALSA) or `paplay` (PulseAudio) must be installed separately; the game degrades to silence if neither is present.

## VII. Future Scope

- 1) Multiplayer mode: extend the WebSocket server to handle concurrent clients with garbage-line mechanics and a shared leaderboard.
- 2) T-spin detection: implement 4-corner T-spin scoring using the existing `last_was_rotate` flag infrastructure.
- 3) Sprint mode: timed 40-line clear challenge demonstrating advanced process scheduling (deadline-driven tick rate).
- 4) Replay system: record timestamped inputs to file (file system extension) and replay them deterministically via the seeded LCG.
- 5) WebRTC signaling: peer-to-peer multiplayer over the internet without a relay server.
- 6) Persistent sessions: token-based session tracking so a web player’s game survives a server restart.
- 7) DAS/ARR in terminal mode: port the 170 ms / 50 ms repeat system from the web client to the `keyboard.c` raw-mode input path.

## VIII. Conclusion

The Tetris OS Simulator successfully demonstrates all seven fundamental OS module categories through a practical, interactive application. By replacing every standard library function with a custom implementation, the project provides a concrete, hands-on understanding of how operating systems manage processes, memory, I/O devices, file systems, and error conditions at the lowest level. The dual-mode architecture (terminal + WebSocket) showcases hardware abstraction principles, while the unified leaderboard file, real-time JSON protocol, and automatic reconnection demonstrate file system and networking implementations working together. Both build targets compile with zero warnings under `-Wall -Wextra`, comprehensive edge-case testing confirms robust error handling, and the full commit history at <https://github.com/Mehak261124/Tetris> documents the incremental development of each module.

## References

- [1] ANSI/ISO 9899:1999, Programming Languages — C, International Organization for Standardization, 1999.
- [2] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF RFC 6455, Dec. 2011. [Online]. Available: <https://tools.ietf.org/html/rfc6455>
- [3] D. Eastlake and P. Jones, “US Secure Hash Algorithm 1 (SHA-1),” IETF RFC 3174, Sep. 2001.
- [4] The Tetris Company, Tetris Guideline, 2009. [Online]. Available: [https://tetris.wiki/Tetris\\_Guideline](https://tetris.wiki/Tetris_Guideline)
- [5] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ: Pearson, 2014.
- [6] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Upper Saddle River, NJ: Pearson, 2017.
- [7] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [8] React Team, “React Documentation,” Meta Open Source, 2024. [Online]. Available: <https://react.dev>
- [9] IEEE Std 1003.1-2017, The Open Group Base Specifications Issue 7, IEEE, 2017.

- [10] D. E. Knuth, The Art of Computer Programming, Vol. 2: Seminumerical Algorithms, 3rd ed. Boston, MA: Addison-Wesley, 1997.